



MACHINE LEARNING

Which Regularizer Should You Actually Use? Lessons from Simulations

A practitioner's decision framework for Ridge, Lasso based on three quantities you can compute before

Ahsaas Bajaj

May 2, 2026 14 min read

LATEST

EDITOR'S PICKS

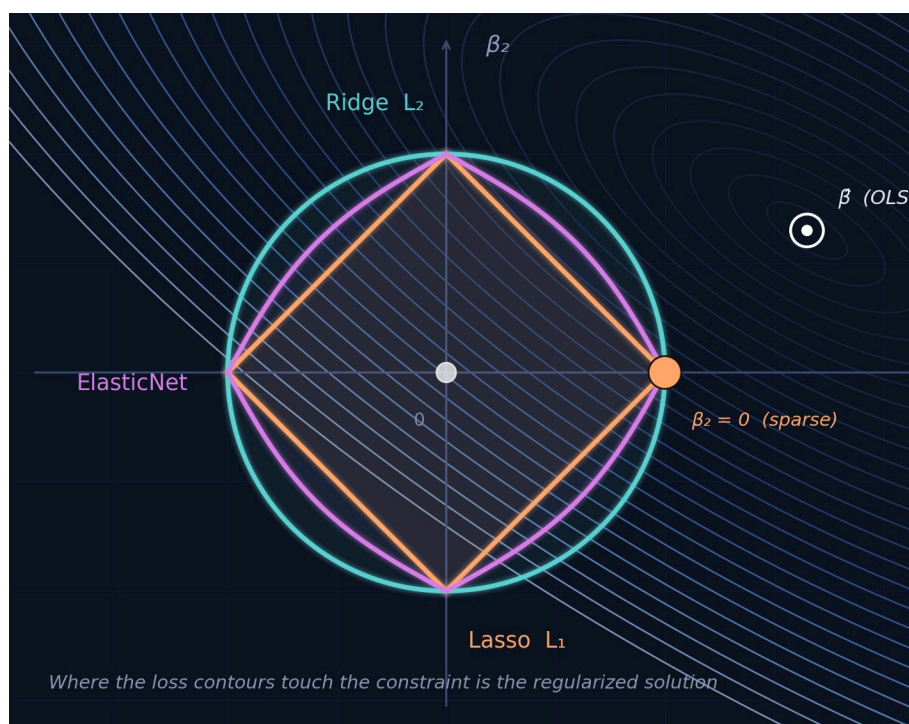
DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article



Authors: Ahsaas Bajaj and Benjamin S Knight

Ridge, Lasso, or ElasticNet? We ran 134,400 simulations in real production ML models to find out. The answer depends on what you're optimizing for, and on a single diagnostic you can compute before fitting a model.

If you've ever trained a linear model in scikit-learn, you've faced this question: RidgeCV, LassoCV, or ElasticNetCV? Maybe you defaulted to whatever a tutorial recommended. Maybe a colleague had a strong opinion. Maybe you tried all three and picked whichever gave the best cross-validation score.

We wanted to replace intuition with empirical decision-making.

We ran 134,400 simulations across 960 configurations in a 10-dimensional parameter space, varying sample size, multicollinearity, signal-to-noise ratio, coefficient regularization, and two more parameters. We benchmarked four regularization frameworks (Ridge, Lasso, ElasticNet, and Poisson) across the three objectives:

1. **Predictive accuracy** (test RMSE)
2. **Variable selection** (F1 score for recovering true set)
3. **Coefficient estimation** (L2 error vs. true coefficients)

Our simulation ranges aren't arbitrary. They're **real-world production ML models** from Instacart, demand forecasting, conversion prediction, and fraud detection intelligence. The regimes we tested reflect conditions you actually encounter in practice.

This post distills the practical guidance from our analysis into a decision framework you can use on your next project. Whether you're a Data Scientist or MLE choosing a regularizer, this is for you.

The Headlines

Before we get into the details:

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

- **For prediction, it barely matters.** Ridge, Lasso, and ElasticNet differ by at most 0.3% in median RMSE. No hyperparameter achieves even a small effect size for RMSE differences among them. This only holds with adequate training data (> 78 observations per feature).
- **For variable selection, it matters enormously, especially under multicollinearity.** Lasso’s recall collapses to 0.18 under high condition numbers with low signal-to-noise ratio. ElasticNet maintains 0.93.
- At large sample-to-feature ratios ($n/p \geq 78$), Ridge and ElasticNet become **interchangeable**. Use Ridge; it’s the fastest.
- **Post-Lasso OLS should be avoided when c**oncerned with **RMSE**. It’s the only method that consistently overfits and it does so on every objective we measure.

LATEST

EDITOR’S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

What We Tested and Why

Submit an Article

Our simulation framework varies seven hyperparameters simultaneously:

Parameter	Values Tested
Sample size (n)	100 · 1,000 · 10,000 · 100,000
Features (p)	64 · 128
Condition number (κ)	$\sim 10^1$ (well-conditioned) · $\sim 10^4$ (ill-conditioned)
Signal-to-noise ratio	0.04 · 0.20 · 1.0
Sparsity	0% (dense) · 15% (sparse)
β distribution	Uniform · Normal
Rank ratio	0.5 · 1.0

Table 1: We simulated a hyperparameter space of 960 combinations.

We ran each of the 4 regularization frameworks across 960 hyper-parameter configurations, each using 35

a total of 134,400 simulations. For every simulation we logged the test RMSE, F1 score (precision and recall for recovering the true support of β), and coefficient L2 error.

To measure *what drives the differences between methods*, we used **omega-squared (ω^2)** from one-way ANOVA, an effect size that tells us what proportion of variance in performance gaps is explained by each parameter. This goes beyond asking “which method wins” to understanding *why* it wins, across different conditions.

Here’s what this means in practice: **most of the differences in method performance are driven by things you can control when fitting a model.** You know n and p . You can control the condition number κ with `numpy.linalg.cond(X)`. You know the important latent parameter, SNR, has a free distribution. You can tune the regularization strength α that LassoCV selects. You can signal low signal; low α signals strong signal. This is key to this.

[LATEST](#)
[EDITOR’S PICKS](#)
[DEEP DIVES](#)
[NEWSLETTER](#)
[WRITE FOR TDS](#)
[Sign in](#)
[Submit an Article](#)

Finding 1: For Prediction, Just Use |

This is the most important finding for the large majority of practitioners.

Ridge, Lasso, and ElasticNet are nearly interchangeable for prediction. Across all 33,600 simulations per method, the median test RMSE differs by at most 0.3%. Our analysis confirms this: no single hyperparameter setting shows a small effect size ($\omega^2 \geq 0.01$) for RMSE difference between the three methods. Every pairwise comparison is not significant ($p > 0.02$).

For practitioners who only care about accuracy, the near-equivalence is itself the finding. Regularizer choice matters far less than sample size.

For Prediction, Regularizer Choice Barely Matters Given Sufficient n

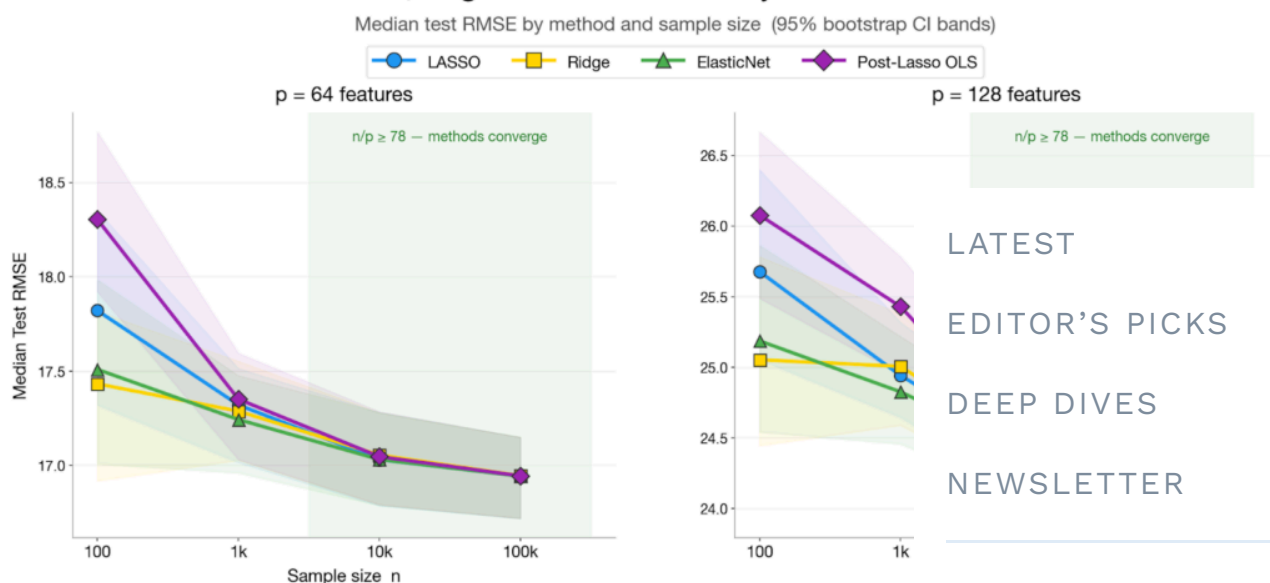


Figure 1: Differences in test RMSE become trivial given sufficient

Sign in

So why Ridge? Computational efficiency. Ridge form solution for each candidate α , making it faster than the alternatives (compare Ridge's median seconds to Lasso's median runtime of 9 seconds to ElasticNet's median runtime of 48 seconds).

Submit an Article

The Speed Tax: ElasticNet Can Be Orders of Magnitude Slower

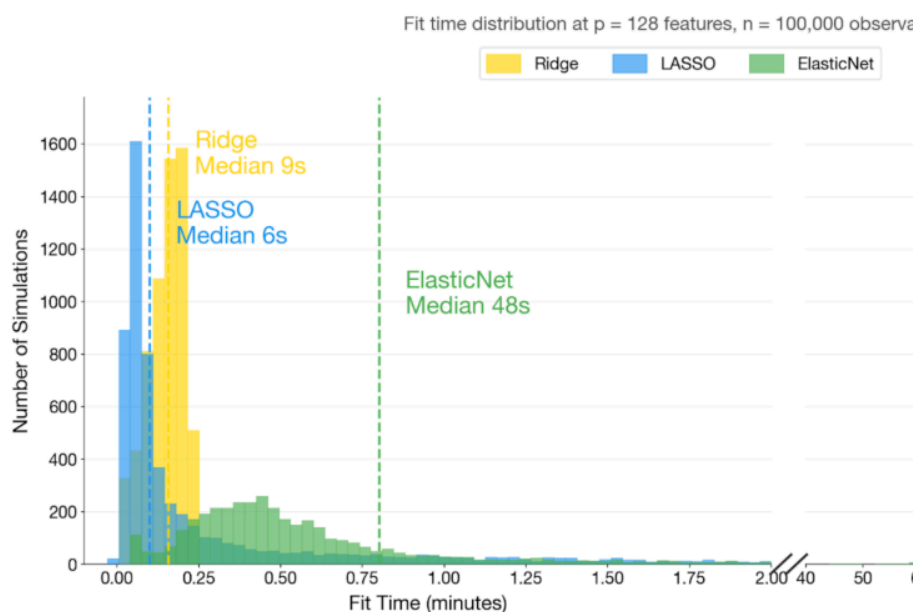


Figure 2: Users should expect a minimum of a 5X increase in runtimes when opting for ElasticNet over Ridge or Lasso.

ElasticNet's overhead stems from its joint grid search over α and the L1 ratio ρ . The 167–219× mean overhead we measured is specific to our 8-value L1 ratio grid. A coarser 3-value grid would reduce this proportionally. Even worse, when the coefficient distribution is approximately uniform, Lasso can take over an hour to converge (see the right-side of the bin). This overhead buys you a median RMSE improvement of 0.04% over Ridge, a margin that's negligible in

Caveats

At the smallest sample size we tested ($n = 100$), ElasticNet beat Ridge by 5–15% in very specific instances (~ 1.0). At low SNR, Ridge is actually marginally better for localized observations at the extreme of our simulated systematic trends.

One more note: LassoLars wasn't part of our review, but the LARS algorithm computes the entire L1 regularization path analytically in a single pass, potentially matching Ridge's closed-form speed. However, LARS is known to be numerically unstable under high collinearity conditions ($\kappa > 10^4$) that characterize production ML feature sets. This is precisely the regime where our strongest findings apply.

Bottom line for prediction: Default to RidgeCV. Model fit matters far more than regularizer choice. But ρ and α are the only objective worth optimizing. When variable importance or coefficient accuracy matters, especially under non-normality, the story changes dramatically.

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Finding 2: For Variable Selection, ElasticNet Is the Safe Default

Here method choice actually matters. Variable selection, the task of identifying which features truly contribute to the outcome, is the objective most sensitive to the regularizer, and where getting it wrong carries the steepest cost.

What Drives the Differences

From our ANOVA decomposition of pairwise F1

Factor	Avg ω^2
Sample size (n)	0.308
SNR ($latent$)	0.065
β distribution ($latent$)	up to 0.112 (ElasticNet vs. Ridge)
Condition number (κ)	0.013

Table 2: Sample size is the most salient predictor of difference

Sample size dominates overwhelmingly. But in small- n regime ($n/p < 78$), the condition number become the primary differentiators.

High Multicollinearity ($\kappa > \sim 10^4$): Do Not Use Lasso

This is one of the most robust findings in the literature, and it's directly relevant to production ML. **Seven of 10 surveyed operate in the high- κ regime.** If your data is moderately correlated (which they almost certainly are in an engineered feature set), this finding applies to you.

At high κ with low SNR:

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

- **Lasso recall: 0.18** (it misses 82% of true features)
- **ElasticNet recall: 0.93** (it catches 93% of true features)

That's a **5× recall advantage for ElasticNet**. The mechanism is well-known. When features are highly correlated, Lasso arbitrarily picks one from each correlated group and zeros the rest. ElasticNet's L2 penalty component, the "grouping effect" described by Zou and Hastie (2005), keeps cor

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

Our simulations show this isn't a corner case. differences ($\Delta F1$ of 0.50–0.75) concentrate squ columns at $n = 100$ and $n = 1,000$. This is the c production.

Low Multicollinearity ($\kappa < \sim 10^2$): Still Default to

You might expect Lasso to finally shine at low least not universally. Even at low κ , Lasso's rec sensitive to the signal-to-noise ratio (see belo

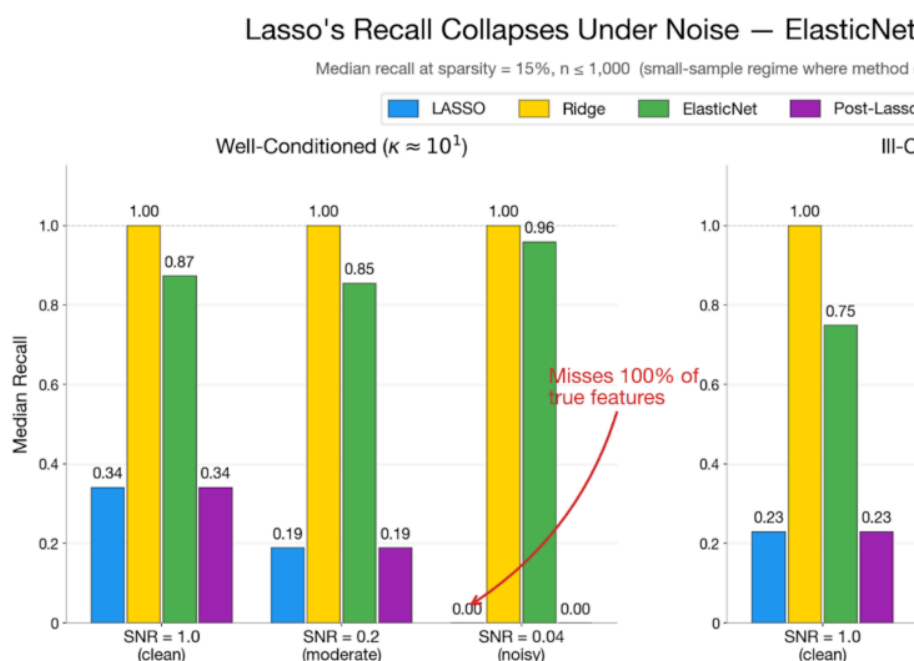


Figure 3: ElasticNet's use of the L2 norm protects against the recall collapse.

ElasticNet maintains recall ≥ 0.91 regardless of SNR, even at low κ . Lasso is only competitive when *both* SNR is high *and* the true model is genuinely sparse. Since you typically don't know SNR in advance, ElasticNet is the safer bet.

The Ridge Surprise

We didn't expect this: Ridge frequently achieves the *highest* F1 scores at small n , despite never performing explicit variable selection. How? Ridge's recall is always 1.0, because it has a nonzero coefficient for every feature, and that perfect recall overwhelms the advantage of sparse methods when those methods' recall collapses under low SNR.

But this isn't genuine variable selection. Ridge has a nonzero coefficient for every feature. If you need a sparse model, Ridge doesn't help. Combining Ridge with *post hoc* permutation importance is a natural extension, but we didn't evaluate it here.

Variable Selection: Summary

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Which Regularizer Should You Use for Variable Selection?

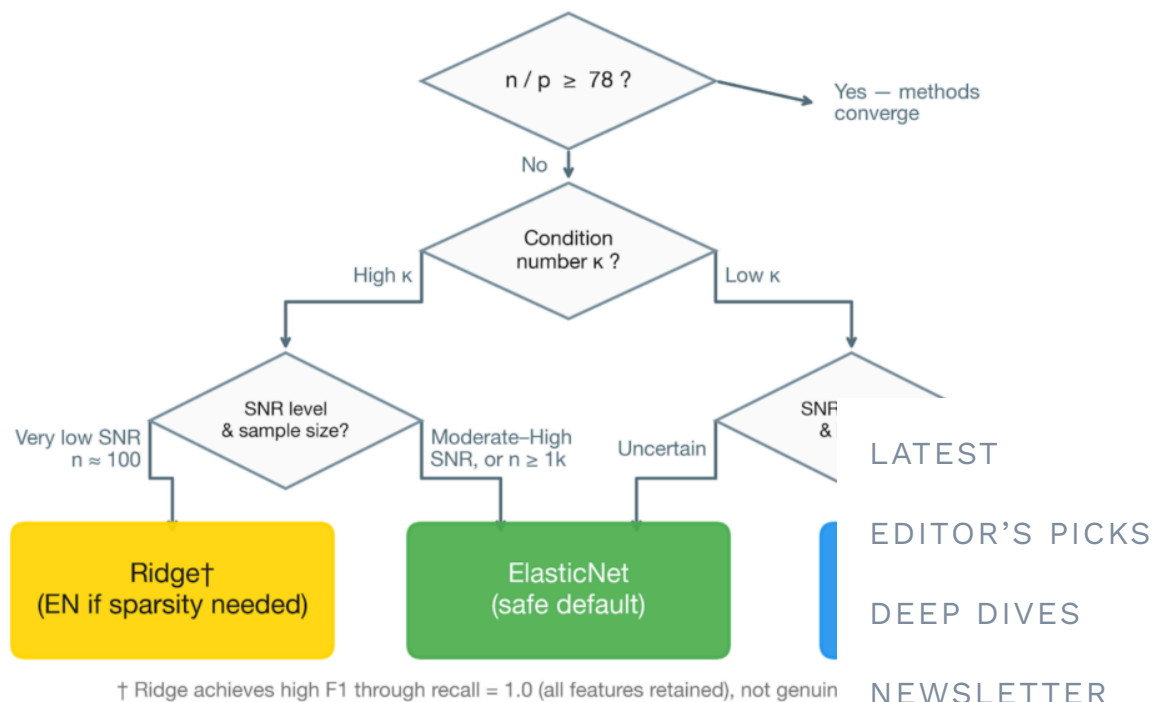


Figure 4: ElasticNet is the safe choice when the researcher cannot

Bottom line for variable selection: ElasticNet is the safe default. Lasso only earns its place when κ is low and you have domain reason to believe the true

Sign in

Submit an Article

Finding 3: For Coefficient Estimation

K

When the goal is recovering accurate coefficient estimates, interpretability or causal inference, the condition number becomes the key branching variable. Ideally we would observe the distribution of the true β coefficients, but we cannot observe it. In contrast, κ can be measured directly. ElasticNet dominates regardless of sparsity. At the optimal method depends on whether the true model is dense. Sample size changes the *magnitude* of the coefficients, not their *direction*.

High κ ($> \sim 10^4$): Use ElasticNet. It achieves 20–40% lower L2 coefficient error than Lasso, and holds a consistent edge over Ridge regardless of sparsity level.

Low κ ($< \sim 10^2$): Branch on your domain knowledge about sparsity.

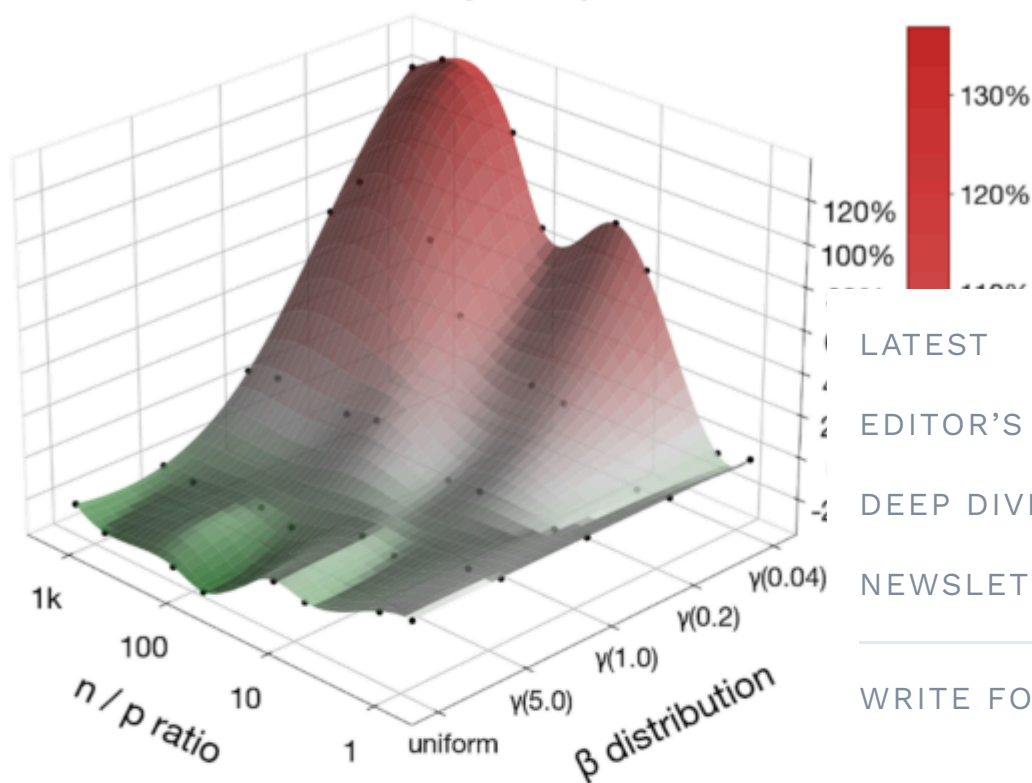
- *Sparse domain* (genomics, text classification, sensor arrays):
Lasso or ElasticNet
- *Dense domain* (engineered feature sets, deconvolution models): Ridge

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Relative Δ L2 Coefficient Error: (Ridge – Lasso) / Lasso

Results marginalized over SNR, sparsity, rank ratio

Well-Conditioned ($\kappa \approx 10^1$)



LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

Ill-Conditioned ($\kappa \approx 10^{5-6}$)

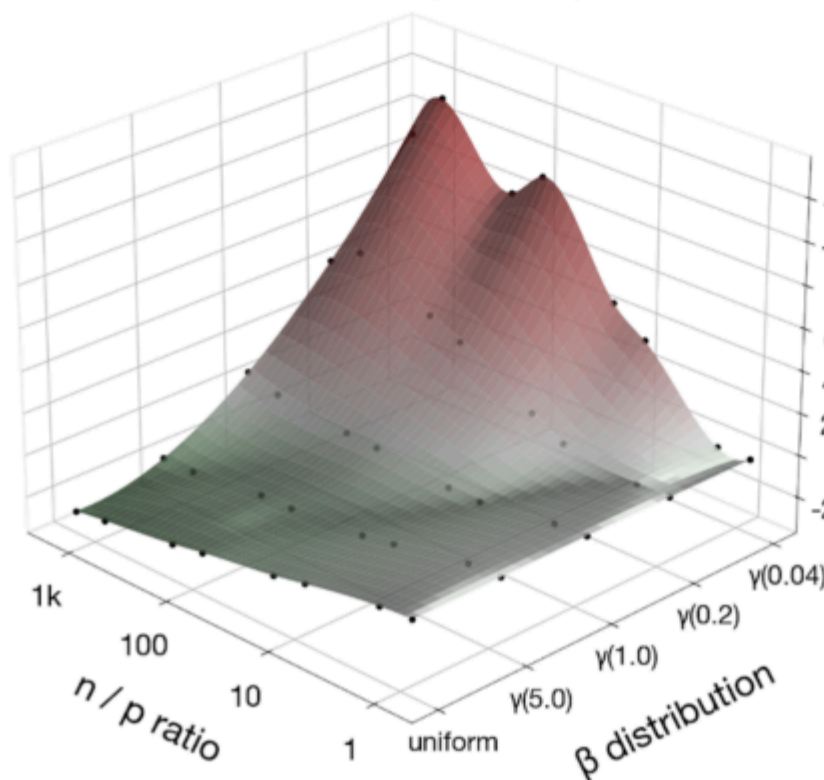


Figure 5: Ridge's performance advantage over Lasso / ElasticNet fades quickly as the n / p ratio increases, while a well-conditioned eigenspace further advantages Lasso / ElasticNet.

All regimes: Avoid Post-Lasso OLS. It shows higher coefficient L2 error than standard Lasso across the entire simulation grid. The unpenalized OLS refit amplifies first-stage selection errors. This is the scenario where you'd hope the two-stage procedure helps, and it doesn't.

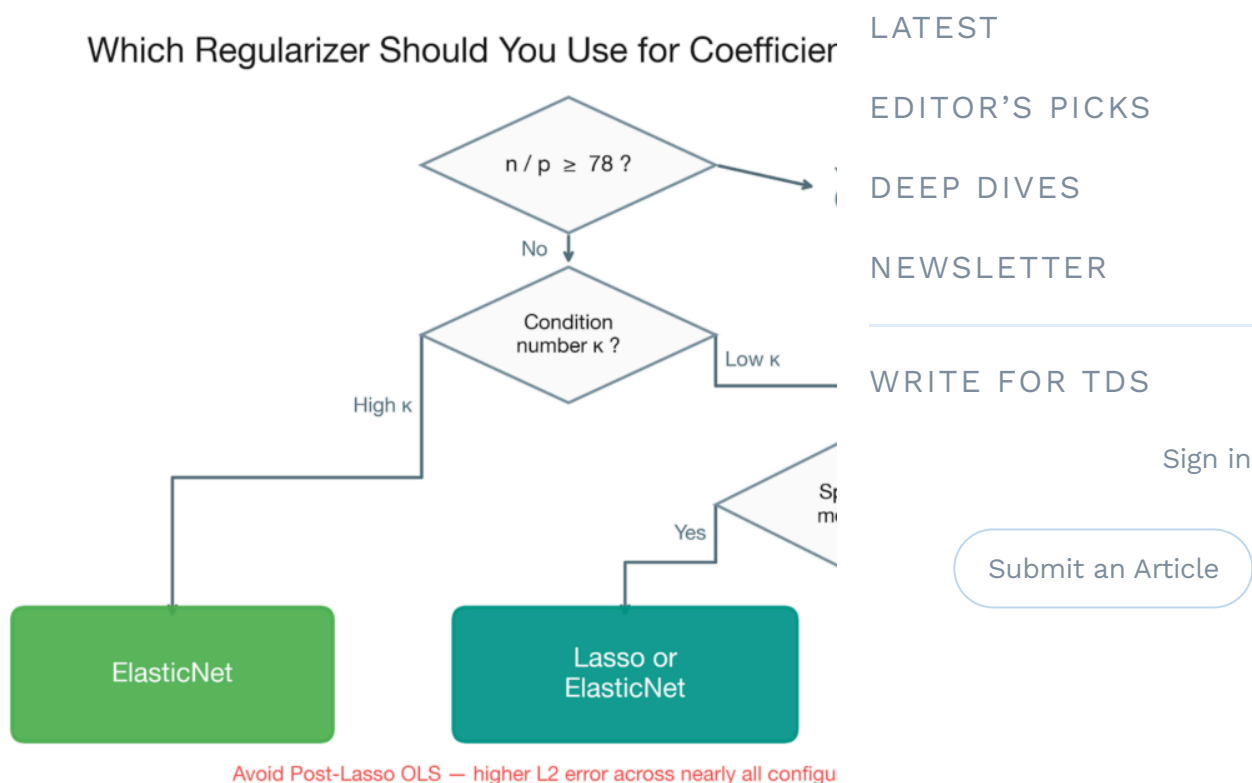


Figure 6: When the goal is coefficient estimation, Ridge becomes

Bottom line for coefficient estimation: ElasticNet is domain-dependent at low κ , never Post-Lasso

A Practitioner's Decision Guide

All of the findings above distill into a decision tree exclusively on quantities you can compute when fitting a single model: the sample-to-feature ratio n / p and condition number κ (via `numpy.linalg.cond(X)`),

discrimination is needed, the regularization strength α elected by a quick LassoCV run as a proxy for the latent SNR.

The full flowchart is available in our paper (Figure 7). Here, we walk through the logic as a decision tree.

The under-determined regime

If your feature count exceeds your sample size, you're in the under-determined regime. Lasso's α frequently serves as the upper boundary of the search grid here, and it's a good idea to **Default to Ridge or ElasticNet** for all objective functions, but proceed with caution.

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)

The large-sample regime

If $n/p \geq 78$, you're in the large-sample regime where methods converge. Performance gaps vanish across preselection, and coefficient estimation simultaneously.

[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Use RidgeCV. It's the fastest method by a wide margin with no accuracy penalty. If you specifically need interpretability, ElasticNetCV or LassoCV are fine at this ratio. The choice among them is immaterial.

The regime where choice matters

Below $n/p = 78$ is where method choice matters. The best regularizer depends on what you're optimizing for.

If prediction is your priority: Use RidgeCV. The differences among the core three methods are too small to matter for complexity or compute. One narrow exception is low SNR (~ 1.0), where ElasticNet offers a detectable improvement.

regardless of κ ; at $n \approx 100$ with very low SNR, Ridge is marginally preferred. In either case, the margin is modest relative to the improvement available from increasing sample size.

If variable selection is your priority: Branch on the condition number.

- **$\kappa > \sim 10^4$** (high multicollinearity): Use ElasticNetCV. This is among the strongest recommendations in the literature. At moderate-to-high SNR (or $n \geq 100$), Ridge is clearly preferred, with F1 advantages over Lasso of $\Delta F1$ of +0.75. At very low SNR with $n \approx 100$ (saturated CV-elected α), Ridge achieves the best F1 only through perfect recall (retaining all features), while ElasticNet achieves perfect recall through genuine variable selection. If you need an interpretable model even in this corner, ElasticNet remains the best option and still vastly outperforms Lasso.
- **$\kappa < \sim 10^2$** (well-conditioned): An important caveat: **do not default to Lasso even at low κ .** Lasso's performance drops sharply at lower SNR levels regardless of n , while ElasticNet maintains recall ≥ 0.91 across all SNR levels. ElasticNet is the safe default here. If you run a quick LassoCV and inspect the elected features, you're saturated at the boundary, you're in a low-SNR regime. ElasticNet provides the best F1 (though not through aggressive sparsification). If α is moderate, stick with ElasticNet. Low κ and domain expertise suggests sparsity is viable.

If coefficient estimation is your priority: Branch on the condition number.

- **$\kappa > \sim 10^4$:** ElasticNetCV dominates regardless of SNR.

[LATEST](#)
[EDITOR'S PICKS](#)
[DEEP DIVES](#)
[NEWSLETTER](#)
[WRITE FOR TDS](#)
[Sign in](#)
[Submit an Article](#)

- $\kappa < \sim 10^2$: Use domain knowledge. Sparse model $\rightarrow$ Lasso.
Dense model $\rightarrow$ Ridge.

The α Diagnostic: A Free SNR Proxy

The one latent parameter that matters for fine-grained decisions, signal-to-noise ratio, can be approximated at zero additional cost. When scikit-learn's LassoCV fits your data, it reports the elected α . This value is inversely related to the underlying SNR: high α signals weak signal, low α signals strong signal.

Our simulations provide direct empirical confirmation that the highest elected α values (approaching 10^4 – 10^5) occur exclusively in small- n , low-SNR configurations.

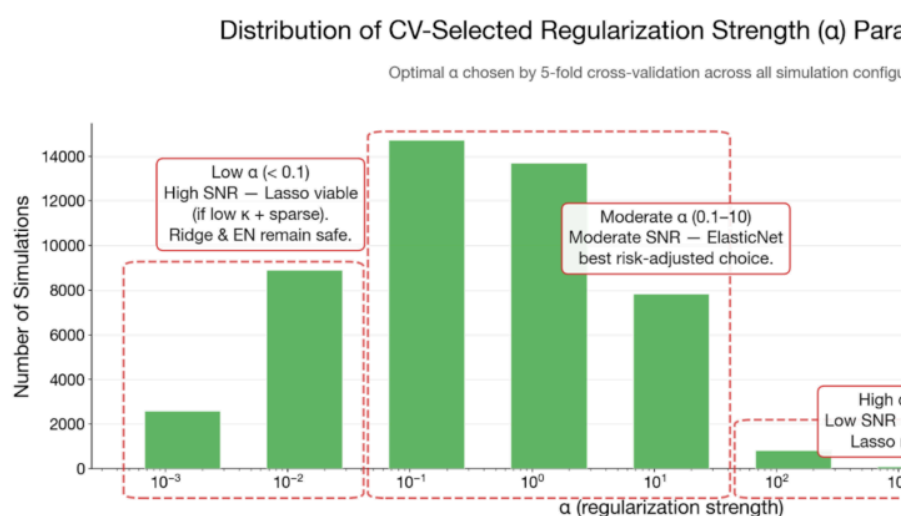
[LATEST](#)
[EDITOR'S PICKS](#)
[DEEP DIVES](#)
[NEWSLETTER](#)
[WRITE FOR TDS](#)
[Sign in](#)
[Submit an Article](#)


Figure 7: The regularization parameter α can be a useful proxy for SNR.

These thresholds are approximate heuristics derived from the simulation grid; they'll vary with feature scaling and data characteristics. Treat them as guidelines, not strict rules.

In All Uncertain Cases

When you're unsure about SNR, unsure about sparsity, or operating in the intermediate- k range we didn't directly test: **ElasticNet is the default that won't burn you**, and **Post-Lasso OLS should be avoided**.

The Meta-Finding: Sample Size Trumps Everything

One takeaway matters more than any method-**increasing your sample-to-feature ratio does** | **objective than any regularizer choice**.

Sample size is the dominant driver of perform across all three metrics ($\omega^2 = 0.308$ for F1, a $\log \times$ SNR interaction is the strongest two-way int comparisons ($F = 569, p < 0.001$). Signal-to-no precisely when samples are scarce. And at n/p choice becomes irrelevant entirely.

If you're spending days tuning your regularizer be growing your training set, you're optimizing

Quick Reference

Objective	Default	When to Switch
Prediction	RidgeCV	ElasticNet only if $n \approx 100$ + high SNR (marginal gain)
Variable Selection	ElasticNetCV	Lasso only if: low κ + high SNR + sparse domain
Coef. Estimation	ElasticNetCV (high κ) / Ridge (low κ , dense)	Lasso at low κ + sparse

Note: When $n/p \geq 78$, all methods perform equivalently. Default to RidgeCV (fastes

Table 3: The most appropriate regularizer is determined by both the nature the research objective.

Putting It Into Practice

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

The simulation framework is a reusable harness. We capped sample sizes at 100k observations for compute reasons, but the grid still spans the n/p inflection point where regularizer performance shifts. We're extending it now to newer regularizers (Adaptive Lasso, SCAD, MCP) and intermediate κ levels.

To apply this framework to your next project, compute three quantities before you fit anything: the sample-to-feature ratio (n/p), the condition number (κ), and if you're in the high SNR regime, a quick LassoCV α as your SNR proxy. I've put together a decision guide above based on your primary objective.

If $n/p \geq 78$, use Ridge and spend your tuning budget on λ .
If $n/p < 78$ and κ is high, use ElasticNet and do a grid search.
It's the only scenario where the choice requires a grid search. Low κ with small n , and even there, ElasticNet is the answer.

The full paper, including all appendix figures, Appendix A, and the consolidated decision flowchart, is available on the paper's GitHub page.

Ahsaas Bajaj is a Machine Learning Tech Lead at Amazon.
Benjamin S Knight is a Staff Data Scientist at Amazon.

All images were created by the authors.

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

WRITTEN BY

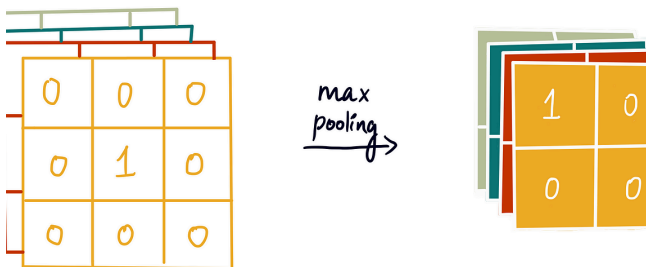
Ahsaas Bajaj

[See all from Ahsaas Bajaj](#)[Data Science](#)[Deep Dives](#)[ML Engineering](#)[Model Optimization](#)[Regularization](#)**Share This Article**[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)

Towards Data Science is a community publication where you can publish your insights to reach our global audience through the TDS Author Payment Program.

[Write for TDS](#)[Submit an Article](#)

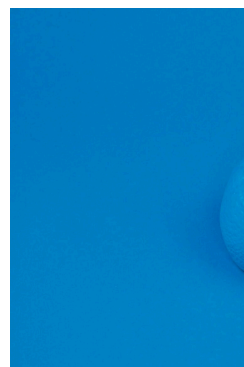
Related Articles



ARTIFICIAL INTELLIGENCE

Implementing Convolutional Neural Networks in TensorFlow

Step-by-step code guide to building a Convolutional Neural Network

[Ahsaas Bajaj](#)

DATA SCIENCE

Hands-on Time Series Detection using Python

Here's how to use Python to detect signals in time series data.

August 20, 2024 6 min read

Piero Paialunga

August 21, 2024 12 min read



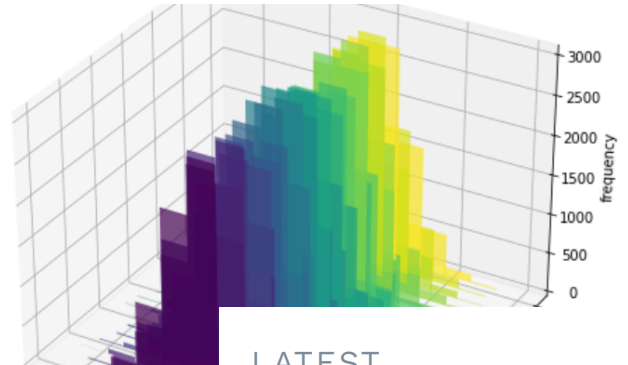
DATA SCIENCE

Back To Basics, Part Uno: Linear Regression and Cost Function

An illustrated guide on essential machine learning concepts

Shreya Rao

February 3, 2023 6 min read



DATA SCIENCE

Must-Know Bivariate No Explained

Derivation and powerful conce

Luigi Battistoni

August 14, 2024

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

[Submit an Article](#)

DATA SCIENCE

Our Columns

Columns on TDS are carefully curated collections of posts on a particular idea or category...

TDS Editors

November 14, 2020 4 min read



DATA SCIENCE

Optimizing Multi-Armed Campaigns v

With demos, ou video

Vadim Arzamasov

August 16, 2024

DATA SCIENCE

Back to Basics, Part Tres: 'istic Regression

An illustrated guide to everything you need to know about Logistic Regression

Shreya Rao

March 2, 2023 8 min read



Your home for data science and AI. The world's leading publication for data analytics, data engineering, machine learning, and artificial intelligence professionals.

© Insight Media Group, LLC 2026

Subscribe to Our Newsletter

WRITE FOR TDS · ABOUT · ADVERTISE · PRIVACY POLICY

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article